The Building Block: A Layer of Neurons

A neural network is constructed by stacking layers of neurons. Each layer takes a vector of numbers as input and produces a vector of numbers as output, which is then passed to the next layer.

Inside a Hidden Layer (e.g., Layer 1)

Let's look at a single hidden layer that takes 4 input features (`x`) and has 3 neurons.

- **Neuron Computation:** Each neuron in this layer functions as an independent **logistic regression unit**. It has its own set of parameters (`w` and `b`) and computes its own output value.
- **Neuron 1:**
  - Computes its activation $a_1 = g(\vec{w}_1 \cdot \vec{x} + b_1)$.
  - $g(z)$ is the sigmoid (logistic) function: $g(z) = \frac{1}{1+e^{-z}}$.
- **Neuron 2:**
  - Computes its activation $a_2 = g(\vec{w}_2 \cdot \vec{x} + b_2)$.
- **Neuron 3:**
  - Computes its activation $a_3 = g(\vec{w}_3 \cdot \vec{x} + b_3)$.
- **Layer Output:** The outputs of these three neurons (e.g., [0.3, 0.7, 0.2]) are collected into a vector. This is the **activation vector** of the layer.

New Notation: Layers ✍️

To keep track of parameters and activations in different layers, we use a **superscript square bracket** `[L]` to denote the layer number.

- **Input Layer:** This is considered **Layer 0**. Its output is just the input feature vector $\vec{x}$.
- **First Hidden Layer (Layer 1):**
  - The parameters for the neurons in this layer are $\vec{w}^{[1]}$ and $b^{[1]}$.
  - The output activation vector of this layer is $\vec{a}^{[1]}$.
- **Second Layer (e.g., Output Layer, Layer 2):**
  - The parameters for this layer are $\vec{w}^{[2]}$ and $b^{[2]}$.
  - The output activation of this layer is $\vec{a}^{[2]}$.

The Output Layer (e.g., Layer 2)

The output layer is just another layer that follows the same rules.

- **Input:** It takes the activation vector from the previous layer ($\vec{a}^{[1]}$) as its input.
- **Computation:** It performs its own logistic regression calculation.
  - $z^{[2]} = \vec{w}^{[2]} \cdot \vec{a}^{[1]} + b^{[2]}$
  - $\vec{a}^{[2]} = g(z^{[2]})$
- **Final Output:** The activation of the output layer, $\vec{a}^{[2]}$, is the neural network's final prediction (e.g., 0.84). This represents the probability that $y = 1$.

Final Prediction (Optional Thresholding)

If you need a binary (0 or 1) prediction, you can apply a threshold to the final probability, just like in logistic regression.

- **Rule:**
  - If $\vec{a}^{[2]} \geq 0.5$, predict $\hat{y} = 1$.
  - If $\vec{a}^{[2]} < 0.5$, predict $\hat{y} = 0$.

Building a More Complex Neural Network

We can build a more complex neural network by stacking multiple layers, where the output of one layer becomes the input for the next.

Network Architecture & Layer Counting

- **Example Network:**
  - **Layer 0:** The **Input Layer** ($\vec{x}$)
  - **Layer 1:** Hidden Layer
  - **Layer 2:** Hidden Layer
  - **Layer 3:** Hidden Layer
  - **Layer 4:** The **Output Layer** ($\hat{y}$)
- **Counting Convention:** When we state the number of layers in a network, we count the **hidden layers and the output layer**. We do *not* count the input layer.
  - The example above is a **4-layer neural network**.

Computation Within a Layer (e.g., Layer 3)

Let's look at the computations for **Layer 3**, which takes the activations from Layer 2 ($\vec{a}^{[2]}$) as its input and produces its own activation vector, $\vec{a}^{[3]}$.

If Layer 3 has 3 neurons (or "units"):

- **Neuron 1:** Computes $a_1^{[3]} = g(\vec{w}_1^{[3]} \cdot \vec{a}^{[2]} + b_1^{[3]})$
- **Neuron 2:** Computes $a_2^{[3]} = g(\vec{w}_2^{[3]} \cdot \vec{a}^{[2]} + b_2^{[3]})$
- **Neuron 3:** Computes $a_3^{[3]} = g(\vec{w}_3^{[3]} \cdot \vec{a}^{[2]} + b_3^{[3]})$

The output of this layer is the vector $\vec{a}^{[3]} = [a_1^{[3]}, a_2^{[3]}, a_3^{[3]}]$.

General Formula for a Single Neuron 🧮

We can write a general formula for the computation of a single neuron (unit `j`) in any given layer (`l`):

$$a_j^{[l]} = g(\vec{w}_j^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]})$$

- $a_j^{[l]}$: The activation (output) of the **j-th neuron** in the **l-th layer**.
- $\vec{w}_j^{[l]}$ & $b_j^{[l]}$: The parameters (weight vector and bias) for the **j-th neuron** in the **l-th layer**.

- $\vec{a}^{[l-1]}$: The vector of activations from the **previous layer (l-1)**. This is the input to the current layer.
- g: The **activation function** (so far, we have used the **sigmoid function**).

Consistent Notation

To make this formula work for the very first layer (Layer 1), we introduce one final piece of notation:

- We define the input feature vector, $\vec{x}$, as the "activation" of Layer 0:
  $\vec{a}^{[0]} = \vec{x}$

This allows us to write the computation for Layer 1 as:
$a_j^{[1]} = g(\vec{w}_j^{[1]} \cdot \vec{a}^{[0]} + b_j^{[1]})$

This process of computing the activations layer by layer is the basis for **inference**, or making predictions with a neural network.

Building a More Complex Neural Network

We can build a more complex neural network by stacking multiple layers, where the output of one layer becomes the input for the next.

Network Architecture & Layer Counting

- **Example Network:**
  - **Layer 0:** The **Input Layer** ($\vec{x}$)
  - **Layer 1:** Hidden Layer
  - **Layer 2:** Hidden Layer
  - **Layer 3:** Hidden Layer
  - **Layer 4:** The **Output Layer** ($\hat{y}$)
- **Counting Convention:** When we state the number of layers in a network, we count the **hidden layers and the output layer**. We do *not* count the input layer.
  - The example above is a **4-layer neural network**.

Computation Within a Layer (e.g., Layer 3)

Let's look at the computations for **Layer 3**, which takes the activations from Layer 2 ($\vec{a}^{[2]}$) as its input and produces its own activation vector, $\vec{a}^{[3]}$.

If Layer 3 has 3 neurons (or "units"):

- **Neuron 1:** Computes $a_1^{[3]} = g(\vec{w}_1^{[3]} \cdot \vec{a}^{[2]} + b_1^{[3]})$
- **Neuron 2:** Computes $a_2^{[3]} = g(\vec{w}_2^{[3]} \cdot \vec{a}^{[2]} + b_2^{[3]})$
- **Neuron 3:** Computes $a_3^{[3]} = g(\vec{w}_3^{[3]} \cdot \vec{a}^{[2]} + b_3^{[3]})$

The output of this layer is the vector $\vec{a}^{[3]} = [a_1^{[3]}, a_2^{[3]}, a_3^{[3]}]$.

General Formula for a Single Neuron 🧮

We can write a general formula for the computation of a single neuron (unit `j`) in any given layer (`l`):

$$a_j^{[l]} = g(\vec{w}_j^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]})$$

- $a_j^{[l]}$: The activation (output) of the **j-th neuron** in the **l-th layer**.
- $\vec{w}_j^{[l]}$ & $b_j^{[l]}$: The parameters (weight vector and bias) for the **j-th neuron** in the **l-th layer**.

- $\vec{a}^{[l-1]}$: The vector of activations from the **previous layer (l-1)**. This is the input to the current layer.
- g: The **activation function** (so far, we have used the **sigmoid function**).

Consistent Notation

To make this formula work for the very first layer (Layer 1), we introduce one final piece of notation:

- We define the input feature vector, $\vec{x}$, as the "activation" of Layer 0:
  $\vec{a}^{[0]} = \vec{x}$

This allows us to write the computation for Layer 1 as:
$a_j^{[1]} = g(\vec{w}_j^{[1]} \cdot \vec{a}^{[0]} + b_j^{[1]})$

This process of computing the activations layer by layer is the basis for **inference**, or making predictions with a neural network.